

Travaux dirigés n° 4

Signaux

Exercice 1 (Boucle infinie)

Nous supposons qu'un programme (que nous appellerons le père) crée 3 fils. Il réalise ensuite une action dans une boucle infinie. Les fils exécutent également une boucle infinie.

- 1°) Que se passe-t-il si le père reçoit le signal SIGINT envoyé depuis un autre programme ? Que peut-on dire sur ses fils ?
- 2°) Nous souhaitons que le père arrête sa boucle infinie et poursuive son exécution à la réception du signal SIGINT. Comment faire ?
- 3°) Donnez la portion de code permettant de mettre en place le gestionnaire pour le signal.
- 4°) Comment faire pour arrêter la boucle infinie des fils lorsque le père reçoit le signal SIGINT ?
- 5°) Écrivez cette portion de code.
- 6°) Le comportement est différent lorsqu'on presse la touche CTRL + C (les fils s'arrêtent en même temps que le père). Comment expliquer cela ?

Exercice 2 (Ping-pong)

Nous souhaitons écrire deux programmes qui communiquent via des signaux temps-réel. Le *serveur* se met en attente d'un signal SIGRTMIN + 1. Le *client* génère une valeur aléatoire de 1 à 5 (ou la demande à l'utilisateur) et l'envoie au *serveur*. À son tour, le *serveur* génère une valeur aléatoire de 1 à 5. Si la différence est supérieure ou égale à 2 (en valeur absolue), le serveur perd. Sinon, il renvoie cette valeur au *client* qui génère à son tour une nouvelle valeur. La partie de ping-pong s'arrête lorsque l'un des deux programmes gagne. Ils arrêtent alors leur exécution.

- 1°) Quelles données sont échangées ? Proposez une solution pour gérer la victoire et la défaite.
- 2°) Le PID du *serveur* est affiché au démarrage. Comment peut-il récupérer le PID du *client* ?
- 3°) Comment un programme peut se mettre en attente d'un signal sans *attente active* ? Précisez les appels systèmes.
- 4°) Donnez le code permettant l'attente d'un signal sans utiliser de gestionnaire.
- 5°) Donnez le code permettant d'envoyer un entier.
- 6°) Expliquez le fonctionnement général des deux programmes en précisant les appels systèmes.
- 7°) Au lieu d'envoyer un entier, est-il possible d'envoyer une chaîne de caractères ?

Exercice 3 (Application de log)

Nous souhaitons développer une application de log que nous appellerons le *logger*. Pour communiquer avec lui, les applications (que nous appellerons les *clients*) utilisent des signaux temps-réel et des tubes nommés.

Lorsque le *logger* reçoit un signal `SIGRTMIN+1`, il crée un tube nommé dont le nom est `p_PID.bin` où “PID” est le PID du *client*. S’il reçoit le signal `SIGRTMIN+2`, il lit un message (un timestamp, `time_t` et une chaîne de longueur variable) du client envoyé dans le tube nommé correspondant. Le message est stocké dans un fichier binaire dont le nom est `PID.bin` où “PID” est le PID du *client*.

1°) Proposez une structure en C pour les messages que vous nommerez `message_t`. Un message n’a pas de taille maximale.

2°) Donnez le code de la fonction `message_ecrire(int fd, message_t msg)` qui écrit le message dans le tube nommé dont le descripteur de fichier est passé en paramètre.

3°) Est-ce que la fonction précédente peut être utilisée pour sauvegarder le message dans le fichier ?

4°) Donnez le code de la fonction `int ouvrir_fichier(pid_t pid)`. Si le fichier nommé `X.bin` (où X est le PID) n’existe pas, il est créé. Si le fichier existe déjà, il est simplement ouvert. Dans les deux cas, la fonction retourne le descripteur du fichier correspondant.

5°) Donnez le code de la procédure `void message_sauvegarder(message_t msg, pid_t pid)` qui stocke le message passé en paramètre dans le fichier correspondant (utilisez les fonctions précédentes).

6°) Expliquez le code du *logger* et donnez les appels système nécessaires.

7°) Discutez sur le choix d’avoir utilisé les noms de fichiers `PID.bin` où “PID” est le PID du client qui a envoyé le message.